

SMARTBASKET AI

Personalised Grocery, Nutrition, Budgeting and Meal-Planning Assistant

Product Specification • Interface Blueprint • Database Design • Backend Architecture • AI Logic • Deployment Plan • Coding Roadmap

Hackathon theme	Save time through personalisation
Recommended MVP	Compare products → build basket → generate meals → optimise cost and nutrition
Suggested build	Next.js + TypeScript + Tailwind + Supabase + Vercel + Open Food Facts / curated demo data

Core promise: “Tell us your budget, body goals, allergies and available time. SmartBasket builds the best basket and turns it into meals.”

1. Executive Product Definition

SmartBasket AI is a personalised grocery decision engine. It reduces the time users spend comparing supermarket products, interpreting nutrition labels, planning meals, checking allergies and staying within budget. The platform does not merely display products; it ranks choices for each individual and converts a basket into practical recipes and a weekly plan.

1.1 One-minute pitch

A student enters a \$70 weekly budget, a 140 g daily protein goal, a peanut allergy and a 30-minute cooking limit. SmartBasket compares Coles and Woolworths products, explains which choices give the best protein-per-dollar, warns about unsafe items, builds a basket, suggests meals using those ingredients, and shows the money and planning time saved.

1.2 What makes the concept technically strong

- Personalisation: recommendations change according to goals, allergies, preferences, budget and cooking time.
- Transparent optimisation: every recommendation explains its price, nutrition and convenience trade-offs.
- End-to-end workflow: product discovery, comparison, cart, meal generation, optimisation and weekly planning are connected.
- Measurable impact: the system records products compared, substitutions accepted, estimated planning time saved and basket savings.
- Hackathon feasibility: the core experience can operate using a curated product dataset while the architecture remains ready for external APIs.

1.3 MVP versus future platform

MVP for the hackathon	Later production version
35-60 curated Australian products across two stores	Thousands of products refreshed from licensed retailer feeds
10-20 curated recipes plus AI-generated variations	Recipe generation with validated nutrition calculations
Simulated Google sign-in or Supabase Auth	Full authentication, account recovery and consent controls
Rule-based ranking plus LLM explanations	Optimisation service, behavioural learning and stronger evaluation
Manual or sample prices	Partner APIs, affiliate feeds or authorised data collection

2. Scope, Users and Success Metrics

2.1 Primary users

Persona	Need	Main mode
Budget student	Affordable groceries and quick meals	Budget mode
Gym user	Protein and calorie targets	Gym mode

Busy shopper	Fast planning and short recipes	Convenience mode
Allergy-sensitive user	Clear ingredient and allergen checks	Safety-first mode
Health-conscious shopper	Better nutrient balance and lower sugar/sodium	Health mode

2.2 Product requirements

- The user can create or load a profile in under two minutes.
- The user can search and filter products by store, category, diet and goal.
- The user can compare up to three products side by side.
- The cart updates price and nutrition totals immediately.
- The system can identify unsafe or incompatible products before checkout.
- The system can suggest recipes from cart and pantry items.
- The optimiser can propose substitutions and quantify the difference.
- The system can show an estimated time-saving and monetary-saving result.

2.3 Demo success metrics

Metric	Demo target	How it is measured
Task completion time	< 3 minutes	Start profile to generated basket and meals
Product comparison	2-3 products	Comparison component completed
Budget improvement	\$5-\$15 simulated saving	Original basket minus optimised basket
Meal usefulness	At least 4 unlocked meals	Recipes matching current cart
Safety	At least 1 visible allergy block	Conflict detected from product allergens
Time saved	30-60 minutes/week	Transparent heuristic explained in the UI

3. Recommended Technical Architecture

3.1 Technology stack

Layer	Recommended choice	Reason
Frontend	Next.js + React + TypeScript	One codebase for pages, server routes and strong component reuse
Styling	Tailwind CSS + shadcn/ui or custom components	Fast polished UI during a hackathon
State	Zustand or React Context	Simple cart/profile state without excessive

		setup
Backend API	Next.js Route Handlers	Avoid a separate server for the MVP
Database	Supabase Postgres	Relational data, authentication, storage and row-level security
Authentication	Supabase Auth with Google OAuth	Fast user sign-in and profile ownership
AI	Claude or OpenAI API through server-side routes	Meal explanations, comparisons and natural-language assistant
Product nutrition data	Curated seed data + Open Food Facts; optional USDA nutrients	Reliable demo plus expandable public data sources
Deployment	Vercel	Fast Next.js deployment, preview URLs and custom domains
Source control	GitHub	Team collaboration and automatic Vercel deployments

3.2 System flow



3.3 Why a website first

Build a responsive web application first. It is faster to distribute, requires no app-store approval, works on laptops and phones, and can later become a Progressive Web App. A native application should only be considered after the core recommendation workflow is proven.

4. Interface Specification - Screen by Screen

4.1 Landing page

Section	Content	Primary action
Hero	Headline, one-sentence benefit and product visual	Build my smart basket
How it works	Profile → compare → optimise → cook	See demo

Benefits	Save time, save money, hit nutrition targets	Start free
Proof metric	Example: 46 minutes and \$12.40 saved	Try demo profile
Disclaimer	General nutrition guidance, not medical advice	Learn more

4.2 Authentication and onboarding

1. Authentication screen: Google sign-in, email sign-in and a large “Use Demo Profile” button.
2. Personal details: name, age, height, weight, household size and activity level.
3. Goals: calorie, protein, carbohydrate, fat and fibre targets. Allow automatic estimates but keep all fields editable.
4. Restrictions: allergies, dietary pattern, disliked foods and custom exclusions.
5. Shopping preferences: weekly budget, stores, maximum cooking time, skill level and kitchen equipment.
6. Summary: show the profile in plain English before saving.

Onboarding UX rules

- Use a progress indicator: Step 1 of 5.
- Keep each step to 4-7 fields.
- Explain why sensitive inputs are requested.
- Make allergy fields visually prominent and require confirmation.
- Do not block the demo if optional health fields are skipped.

4.3 Dashboard

Widget	What it shows	Interaction
Budget card	Weekly budget, basket cost and remaining amount	Open cart
Nutrition rings	Calories, protein, carbs, fat and fibre coverage	View breakdown
Recommended basket	Best starter products for the active mode	Add all
Meal opportunities	Meals currently possible	Open meal planner
Smart insights	Top 3 actions to improve basket	Accept suggestion
Savings	Time and money saved	View methodology

4.4 Shop and search

- Sticky search bar with natural queries such as “cheap high-protein breakfast”.
- Filter drawer: store, category, price, allergy-safe, diet, high-protein and preparation time.
- Sort: personal match, lowest price, unit price, protein-per-dollar and nutrition score.
- Desktop grid of 3-4 cards; mobile list with compact nutrition values.
- Each card must show the recommendation reason, not only a score.

4.5 Product comparison

The comparison page is the strongest visual proof of intelligence. It should highlight winners for price, protein-per-dollar, sodium, fibre and overall personal fit. A short AI explanation should be produced from structured calculations; the model must not invent nutrient values.

4.6 Cart and meal panel

Left side: Smart Cart	Right side: Meal Suggestions
Items, quantities, store and subtotal	Can make now
Live budget progress	Needs one ingredient
Nutrition coverage	Fastest meals
Allergy and nutrient-gap warnings	Highest-protein meals
Optimise basket button	Add missing ingredients

4.7 Recipe details

- Available ingredients in green; missing ingredients in orange.
- Serving selector recalculates quantities, price and nutrients.
- Preparation time, cooking time, difficulty and required equipment.
- Step-by-step method with timers where useful.
- Allergy warning and substitutions.
- Storage instructions and leftover reuse ideas.

4.8 Savings dashboard

Expose the time-saving calculation instead of presenting a mysterious number. Example: 12 products compared × 45 seconds saved per comparison + 15 minutes meal-planning baseline + 5 minutes shopping-list creation - 3 minutes spent in SmartBasket.

5. Product Data and Database Strategy

5.1 Important limitation: supermarket prices

Do not assume Coles or Woolworths offers an unrestricted public product-price API. For the hackathon, use curated demonstration data or obtain data only through a source whose terms permit reuse. Avoid building the MVP around fragile scraping. In a production version, seek retailer partnerships, licensed feeds, affiliate sources or explicit permission.

5.2 Recommended data sources

Source	Use	Limitations / handling
Curated JSON/CSV seed	Exact demo prices, images and products	Label as demo data; fastest and most reliable

Open Food Facts API	Barcodes, ingredients, allergens and nutrition labels	Crowdsourced data; validate fields and coverage
USDA FoodData Central	Generic food nutrient references	US-focused and may not match Australian branded products
Retailer-authorized feed	Current price, availability and promotions	Best production option; requires agreement
User receipt / barcode input	Personal price history and pantry	Requires consent and careful parsing

5.3 Database entities

Table	Purpose	Key fields
profiles	User settings and targets	user_id, budget, calories, protein, cooking_time
allergies	Master list of allergens	id, name, aliases
profile_allergies	User-allergy mapping	profile_id, allergy_id, severity
stores	Retailers	id, name, logo_url
products	Canonical product identity	id, barcode, name, brand, size, category
product_nutrition	Nutrition per 100 g and serving	product_id, kcal, protein, carbs, fat, fibre, sodium
product_allergens	Product-allergy mapping	product_id, allergen_id, may_contain
store_products	Store-specific price and availability	store_id, product_id, price, unit_price, captured_at
recipes	Recipe metadata	id, title, time, servings, instructions
recipe_ingredients	Recipe requirements	recipe_id, product/category, quantity, optional
carts	Current/saved baskets	id, user_id, status
cart_items	Basket products	cart_id, store_product_id, quantity
pantry_items	Items already owned	user_id, product_id, quantity, expiry_date
recommendation_events	Explainability and analytics	user_id, type, input, output, accepted

5.4 Simplified SQL schema

```
create table profiles (  
  user_id uuid primary key references auth.users(id) on delete cascade,  
  display_name text,  
  weekly_budget numeric(10,2),  
  calorie_target integer,  
  protein_target numeric(8,2),  
  carb_target numeric(8,2),
```

```
fat_target numeric(8,2),
fibre_target numeric(8,2),
max_cooking_minutes integer,
created_at timestamptz default now()
);

create table products (
  id uuid primary key default gen_random_uuid(),
  barcode text unique,
  name text not null,
  brand text,
  category text,
  package_grams numeric,
  image_url text,
  ingredients text
);

create table store_products (
  id uuid primary key default gen_random_uuid(),
  product_id uuid references products(id) on delete cascade,
  store_id uuid references stores(id),
  price numeric(10,2) not null,
  unit_price_per_100g numeric(10,2),
  available boolean default true,
  captured_at timestamptz default now()
);

create table cart_items (
  id uuid primary key default gen_random_uuid(),
  cart_id uuid references carts(id) on delete cascade,
  store_product_id uuid references store_products(id),
  quantity integer check (quantity > 0)
);
```

5.5 Seed-data process

- 7. Create a spreadsheet with one row per store-product combination.
- 8. Normalise nutrition to per 100 g and retain the original serving size.
- 9. Assign stable IDs and product categories.
- 10. Add allergen tags and dietary labels manually for the demo.
- 11. Import CSV into Supabase or convert it to TypeScript seed objects.
- 12. Run validation: no negative prices, required names present, nutrients numeric and duplicated barcodes flagged.

6. Backend and API Specification

6.1 Recommended API routes

Route	Method	Purpose
/api/products	GET	Search and filter products
/api/products/[id]	GET	Product detail
/api/compare	POST	Calculate comparison and explanation
/api/cart/summary	POST	Calculate basket cost and nutrients
/api/recommendations/products	POST	Rank products for the profile

/api/recommendations/recipes	POST	Find recipes matching cart and pantry
/api/optimise	POST	Propose substitutions
/api/ai/explain	POST	Generate grounded natural-language explanation
/api/profile	GET/PATCH	Read or update user settings
/api/metrics	POST	Store time and savings events

6.2 Example request and response

```
POST /api/optimise

{
  "profileId": "demo-alex",
  "mode": "balanced",
  "budget": 70,
  "cart": [
    { "storeId": "sp_101", "quantity": 1 },
    { "storeId": "sp_205", "quantity": 2 }
  ]
}

{
  "originalCost": 67.40,
  "optimisedCost": 59.80,
  "estimatedSaving": 7.60,
  "proteinDifference": 12,
  "suggestions": [
    {
      "replace": "Premium rolled oats",
      "with": "Store-brand rolled oats",
      "saving": 2.20,
      "reason": "Similar nutrition and lower unit price"
    }
  ]
}
```

6.3 Server-side AI safety rule

Only send structured product facts to the AI model. Calculate prices, nutrition totals, allergens and ranking scores in code. Ask the model to explain those facts, suggest cooking variations and format responses. Never allow the model to invent product nutrition or declare an item allergy-safe without matching structured allergen data.

7. Recommendation, Nutrition and Optimisation Logic

7.1 Product scoring

```
personalScore =
  nutritionMatch * 0.30 +
  budgetValue * 0.25 +
  safetyCompatibility * 0.20 +
  mealVersatility * 0.15 +
  convenience * 0.10
```

Each component should be normalised to 0-100. Safety is a hard gate: a confirmed allergen conflict should mark the product unsafe and remove it from automatic recommendations, regardless of the weighted score.

7.2 Useful formulas

Metric	Formula / logic
Unit price	$\text{price} \div \text{package grams} \times 100$
Protein per dollar	$\text{total protein in package} \div \text{price}$
Cost per serving	$\text{price} \div \text{servings}$
Weekly target coverage	$\text{basket nutrient total} \div \text{weekly target} \times 100$
Recipe match	$\text{required ingredients available} \div \text{total required ingredients}$
Budget fit	higher score as cost remains below user budget
Time fit	100 when recipe time $\leq$ limit; gradually lower above limit

7.3 Basket optimiser algorithm

- Calculate the current basket totals.
- For each cart item, find products in the same category and suitable dietary group.
- Remove candidates with allergy conflicts or unavailable data.
- Calculate cost, nutrition, recipe coverage and convenience changes.
- Generate candidate swaps.
- Choose swaps according to the active mode while respecting the budget and minimum nutrition constraints.
- Return a small set of explainable recommendations rather than silently changing the cart.

7.4 Simple TypeScript scoring example

```
export function proteinPerDollar(product: Product): number {
  if (product.price <= 0) return 0;
  const servings = product.packageGrams / product.servingGrams;
  const totalProtein = servings * product.proteinPerServing;
  return totalProtein / product.price;
}

export function hasAllergyConflict(
  product: Product,
  userAllergies: string[]
): boolean {
  const tags = [...product.allergens, ...product.mayContain]
    .map(value => value.toLowerCase());

  return userAllergies.some(allergy =>
    tags.includes(allergy.toLowerCase())
  );
}
```

8. AI Features and Prompt Design

8.1 AI jobs suitable for the MVP

- Explain a structured product comparison in plain language.
- Turn matched recipe data into beginner-friendly cooking steps.
- Respond to natural-language commands such as “reduce my basket by \$10”.
- Suggest meal variations using products already selected.
- Summarise why an optimisation is useful.

8.2 AI jobs that should remain deterministic

- Price totals and unit-price calculation.
- Nutrition totals.
- Allergy conflict detection.
- Budget constraints.
- Ranking score calculation.
- Recipe ingredient matching.

8.3 Grounded AI prompt template

```
SYSTEM:
You are SmartBasket AI. Use only the structured facts supplied.
Never invent prices, nutrition values, ingredients or allergen status.
Do not provide medical diagnosis. Clearly state uncertainties.

USER PROFILE:
{profile_json}

PRODUCTS / CART:
{structured_product_json}

CALCULATED RESULTS:
{calculation_json}

TASK:
Explain the best option in no more than 120 words.
Mention price, protein-per-dollar, nutrition trade-offs and any warnings.
Use simple language suitable for a university student.
```

8.4 Output validation

Request JSON output where possible and validate it with a schema. If model output fails validation, retry once with a stricter prompt or fall back to a deterministic template.

9. Project Structure and Coding Specification

9.1 Suggested folder structure

```
smartbasket/
├─ app/
│   ├─ (auth)/login/page.tsx
│   ├─ onboarding/page.tsx
│   ├─ dashboard/page.tsx
│   ├─ shop/page.tsx
│   ├─ compare/page.tsx
│   ├─ cart/page.tsx
│   └─ meals/page.tsx
```

```

├─ pantry/page.tsx
├─ profile/page.tsx
├─ api/
│   ├── products/route.ts
│   ├── compare/route.ts
│   ├── optimise/route.ts
│   └─ ai/explain/route.ts
├─ components/
│   ├── product/
│   ├── cart/
│   ├── recipe/
│   ├── dashboard/
│   └─ ui/
├─ lib/
│   ├── supabase/
│   ├── scoring.ts
│   ├── nutrition.ts
│   ├── optimisation.ts
│   ├── allergens.ts
│   └─ validators.ts
├─ data/
│   ├── products.json
│   └─ recipes.json
├─ types/
│   └─ index.ts
└─ public/

```

9.2 Core TypeScript interfaces

```

export interface UserProfile {
  id: string;
  name: string;
  weeklyBudget: number;
  calorieTarget: number;
  proteinTarget: number;
  carbTarget: number;
  fatTarget: number;
  fibreTarget: number;
  allergies: string[];
  dietaryPreferences: string[];
  preferredStores: string[];
  maxCookingMinutes: number;
}

export interface Product {
  id: string;
  store: "Coles" | "Woolworths";
  name: string;
  brand: string;
  category: string;
  price: number;
  packageGrams: number;
  servingGrams: number;
  caloriesPerServing: number;
  proteinPerServing: number;
  carbsPerServing: number;
  fatPerServing: number;
  fibrePerServing: number;
  allergens: string[];
  mayContain: string[];
  ingredients: string[];
  imageUrl: string;
}

export interface Recipe {
  id: string;
  title: string;
  ingredients: RecipeIngredient[];
  totalMinutes: number;
  servings: number;
}

```

```
instructions: string[];
nutritionPerServing: Nutrition;
}
```

9.3 Environment variables

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=          # server only; never expose to browser
AI_API_KEY=                          # server only
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Never prefix secret keys with NEXT_PUBLIC_. Keep AI calls and privileged database operations inside server routes.

10. Authentication, Privacy and Security

- Use Supabase Auth for Google OAuth and email login.
- Enable Row Level Security so users can only read and modify their own profiles, carts, pantry items and meal plans.
- Store only information required for personalisation.
- Treat allergies and health-related goals carefully; explain why the data is collected and allow deletion.
- Do not store AI API keys in browser code.
- Validate every API request using a schema library such as Zod.
- Rate-limit expensive AI endpoints.
- Sanitise user-entered recipe or pantry text.
- Add a nutrition disclaimer and encourage professional advice for medical dietary needs.

10.1 Example Row Level Security policy

```
alter table profiles enable row level security;

create policy "Users read own profile"
on profiles for select
using (auth.uid() = user_id);

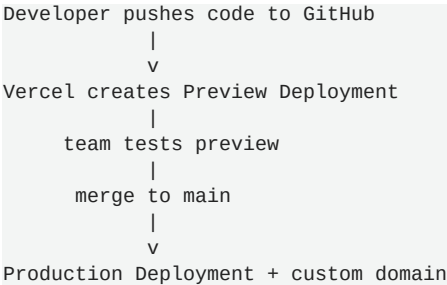
create policy "Users update own profile"
on profiles for update
using (auth.uid() = user_id);
```

11. Domain, Hosting and Deployment

11.1 Domain plan

20. Choose a short name that is easy to say during the pitch, such as smartbasket.ai, smartbasketapp.com or nuticart.app. Check trademark and domain availability before committing.
21. For the hackathon, deploy immediately to the free Vercel URL so the team always has a working demo.
22. Purchase the custom domain from a registrar only after the name is final.
23. Add the domain in Vercel project settings.
24. Update the registrar DNS records using the values shown by Vercel.
25. Set NEXT_PUBLIC_APP_URL to the production address.
26. Update Google OAuth and Supabase redirect URLs.
27. Test HTTPS, login callbacks, mobile layout and all API routes.

11.2 Deployment workflow



11.3 Pre-deployment checklist

- npm run build passes without TypeScript errors.
- No secrets committed to GitHub.
- Database migrations have run.
- Demo account works.
- Fallback product data loads if external APIs fail.
- All product images have fallbacks.
- Mobile cart and comparison screens are usable.
- AI responses have loading, timeout and error states.

12. Step-by-Step Build Plan

Phase	Tasks	Output
0. Team setup (1-2 h)	GitHub repo, Vercel project, task ownership, Figma sketch	Shared development environment
1. UI skeleton (3-4 h)	Navbar, landing, onboarding, dashboard and routes	Clickable polished shell
2. Data model (2-3 h)	Product JSON, recipes, types and validation	Reliable sample dataset
3. Shopping flow (4-6 h)	Search, filters, product cards, details and cart	End-to-end shopping
4. Intelligence (4-6 h)	Scores, allergy checks, comparison and insights	Personalised recommendations
5. Meals (4-5 h)	Recipe matching, missing items and recipe detail	Cart-to-meal experience
6. Optimiser (3-5 h)	Substitution logic and savings display	Strong judging moment
7. Backend/Auth (3-5 h)	Supabase schema, login and persistence	Real accounts and storage
8. AI explanation (2-4 h)	Server route, prompt, schema validation	Natural-language assistant
9. Polish (3-4 h)	Responsive design, animations, empty/error states	Presentation-ready demo
10. Pitch prep (2 h)	Seed demo, rehearse 3-minute journey,	Reliable final pitch

	record backup	
--	---------------	--

12.1 Suggested team allocation

Role	Responsibilities
Frontend 1	Landing, onboarding, dashboard and design system
Frontend 2	Shop, comparison, cart and responsive behaviour
Backend/data	Supabase, schema, product seed, API routes and auth
AI/logic	Scoring, optimisation, recipe matching and LLM prompts
Pitch/product	Demo data, testing, slides, story and time-saving metric

13. Testing and Quality Assurance

Test type	Example
Unit	proteinPerDollar returns the correct value
Unit	allergen aliases match peanuts / groundnuts
Integration	adding cart item updates cost and nutrition
Integration	optimiser never recommends an unsafe product
UI	comparison works on mobile width
Data	all products have positive price and serving size
AI	explanation uses supplied facts and no invented values
Demo	full judge scenario completes with no external API dependency

13.1 Essential edge cases

- Zero or missing serving size.
- Unknown nutrition values.
- Product listed in only one store.
- Budget already exceeded.
- No recipe matches.
- All replacement candidates conflict with allergies.
- AI API unavailable.
- External product API rate-limited or incomplete.

14. Final Demo Script

28. Open the landing page and state the problem: students waste time comparing price, nutrition and meals across separate tools.
29. Load Alex's demo profile: \$70 budget, 140 g protein target, peanut allergy and 30-minute limit.
30. Search for yoghurt and compare two products. Highlight unit price and protein-per-dollar.
31. Open an unsafe snack and show the peanut warning.
32. Add chicken, rice, eggs, yoghurt and vegetables to the cart.
33. Show live cost and nutrition coverage.
34. Open meal suggestions and show meals possible now and meals needing one extra item.
35. Click "Optimise My Basket" and accept two substitutions.
36. Show the final savings: lower cost, similar or improved protein and additional meals unlocked.
37. End on the savings dashboard: estimated planning time and money saved.

14.1 Recommended closing line

"SmartBasket does not just find groceries. It removes the decisions between shopping, nutrition and cooking - turning a personalised goal into a practical basket and meal plan in minutes."

15. Implementation Prompt for Claude

Paste the following prompt into Claude after attaching this PDF. Ask Claude to build one phase at a time and verify that the project runs before continuing.

```
You are the lead full-stack engineer for SmartBasket AI.

Use the attached SmartBasket AI Technical Product Blueprint as the source of truth.

Build a production-quality hackathon MVP using:
- Next.js App Router
- TypeScript
- Tailwind CSS
- Supabase Postgres and Auth
- Zustand for cart/profile state
- Zod validation
- Next.js Route Handlers for backend endpoints
- Vercel-compatible deployment

Rules:
1. First inspect the blueprint and summarise the implementation plan.
2. Create the project in phases. Do not attempt every feature in one response.
3. Start with the folder structure, global layout, routes, TypeScript types, realistic seed products and a demo profile.
4. Every important button must work.
5. All calculations must be deterministic code.
6. The AI model may explain structured results but must not invent prices, nutrition, ingredients or allergen status.
7. Provide complete files with exact paths.
8. After each phase, provide terminal commands and a test checklist.
9. Avoid placeholder pages and broken imports.
10. Keep the design premium, responsive and judge-ready.

Begin with Phase 1:
- initialise the application
- install dependencies
- build the navigation and design system
- create landing, login and onboarding pages
- create the demo profile
- create Product, Recipe, UserProfile and CartItem interfaces
```


- add 20 initial seed products and 8 seed recipes
- verify npm run build succeeds

16. Technical References

Reference	URL	Relevance
Open Food Facts API introduction	https://openfoodfacts.github.io/openfoodfacts-server/api/	Ingredients, nutrition and open product data.
USDA FoodData Central API	https://fdc.nal.usda.gov/api-spec/fdc_api.html	Nutrient reference data and API definitions.
Supabase documentation	https://supabase.com/docs	Postgres database, Auth, Storage and platform overview.
Supabase Auth	https://supabase.com/docs/guides/auth	Authentication and integration with Postgres.
Supabase Database	https://supabase.com/docs/guides/database/overview	Database management and security foundation.
Vercel deployments	https://vercel.com/docs/deployments	Preview and production deployment workflow.
Vercel custom domains	https://vercel.com/docs/domains/working-with-domains/add-a-domain	Domain verification and DNS setup.
Next.js on Vercel	https://vercel.com/docs/frameworks/full-stack/nextjs	Next.js deployment and platform integration.

Data-source and platform terms can change. Review the latest documentation and licence conditions before using external data in a public or commercial release.

Appendix A. MVP Feature Priority

Priority	Feature	Decision
Must	Demo profile and onboarding	Build
Must	Product search, filters and cards	Build
Must	Cart totals and budget progress	Build
Must	Allergy warning	Build
Must	Comparison and recommendation reason	Build
Must	Meal matching and missing ingredients	Build
Must	Basket optimiser and savings	Build
Should	Supabase authentication and persistence	Build if team capacity permits

Should	AI chat assistant	Use one grounded workflow
Could	Receipt scanner	Simulate only
Could	Barcode scanner	Stretch
Do not prioritise	Live checkout and payment	Out of scope
Do not prioritise	Complete live retailer catalogue	Out of scope

Appendix B. Definition of Done

- The application opens from a public URL.
- The full judge demo works without manual database edits.
- The cart and nutrition calculations are correct for seed data.
- Unsafe products are clearly blocked or warned.
- At least one optimisation saves money and explains why.
- At least four recipes respond to cart contents.
- The UI works on a laptop and phone.
- The team has a backup screen recording.

17. Experience Architecture and Interface Upgrade (Version 2)

Design objective: turn the original feature list into a guided, premium experience that feels as simple as choosing a restaurant in Uber Eats, while still delivering powerful grocery, nutrition and recipe intelligence.

17.1 Improved opening concept

The opening should not immediately force the user into a supermarket catalogue. It should first ask what the user wants to achieve. The user may be shopping for a full week, planning a main meal, making a dessert, preparing snacks, or creating a drink. This creates a more human and personalised starting point and ensures the cookbook is not limited to filling meals.

- Weekly groceries: create a complete basket within a weekly budget.
- Main meal: select a cuisine, protein, cooking time or nutritional goal.
- Dessert: suggest baked, chilled, high-protein, low-cost or allergy-safe desserts.
- Snacks: generate portable, study-friendly, gym-friendly or quick snacks.
- Drinks: suggest smoothies, protein shakes, mocktails, iced drinks and hot drinks.
- Use what I have: prioritise pantry and cart ingredients before recommending purchases.

17.2 Recommended top-level navigation

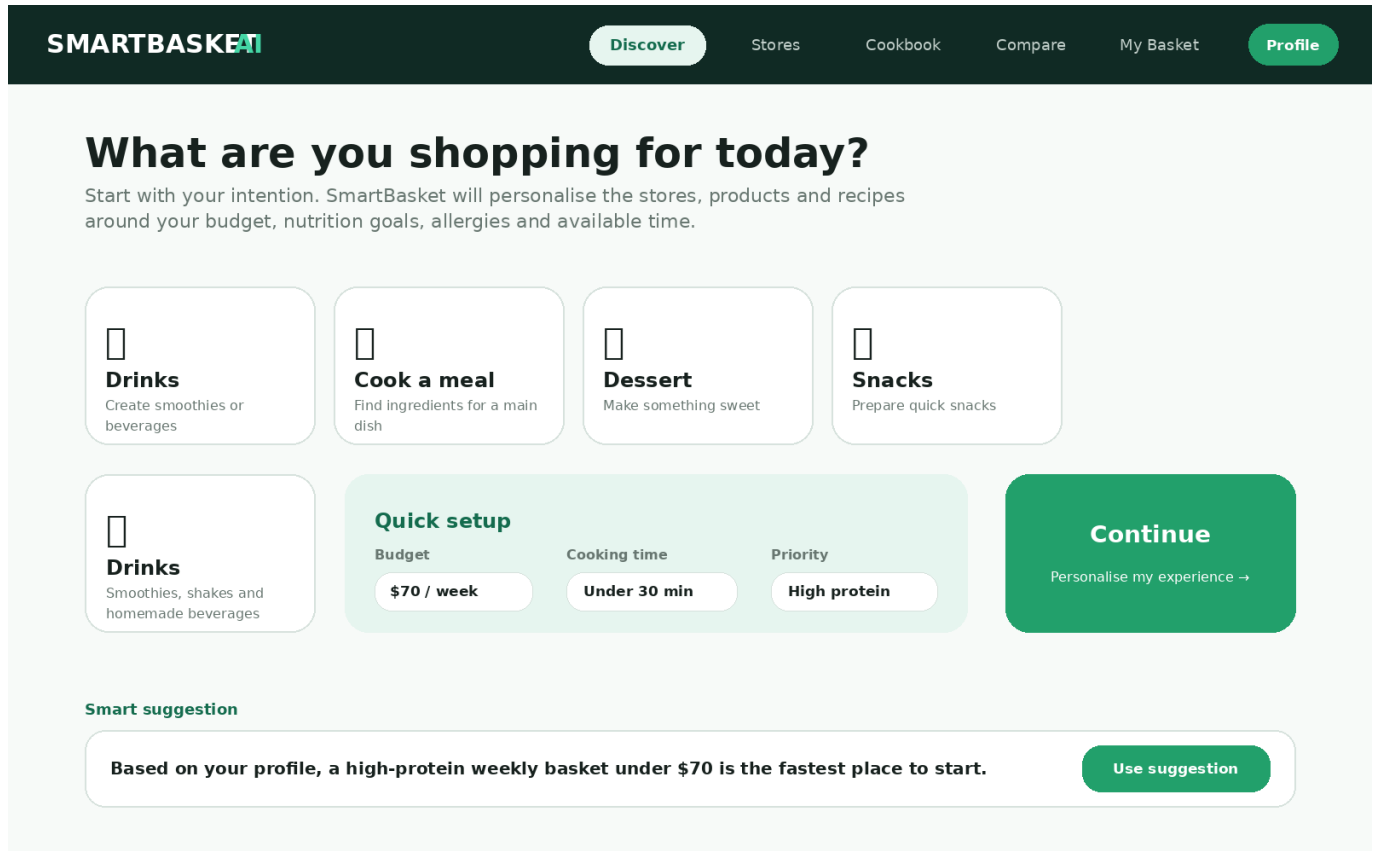
Navigation item	Purpose	Primary outcome
Discover	Intent-based opening and personalised recommendations	Choose what to shop or make
Stores	Select Coles, Woolworths, IGA or future stores	Set comparison strategy
Cookbook	Browse mains, desserts, snacks, drinks and meal prep	Choose recipes and add ingredients
Compare	Compare matching products across stores	Pick the best value and nutrition
My Basket	Review products, budget, nutrients and unlocked meals	Optimise and finalise shopping
Profile	Goals, allergies, preferences and saved plans	Maintain personalisation

17.3 Experience flow

1. User opens SmartBasket and chooses an intention: weekly shop, main meal, dessert, snack, drink or use existing ingredients.
2. User confirms quick constraints: budget, time, dietary restrictions and priority.
3. User selects stores using large, visual store cards similar to a food-delivery application.
4. SmartBasket shows a personalised product feed and relevant cookbook categories.
5. Adding a product updates the basket, nutrition totals and available recipes in real time.
6. The comparison screen explains which store or product is best for price, nutrition and personal goals.
7. The cookbook shows recipes that can be made now and recipes unlocked by one or two extra ingredients.
8. The user accepts substitutions, adds missing ingredients and generates a final shopping plan.

17.4 Opening page specification

- Hero heading: “What are you shopping for today?”
- Six intent cards with clear imagery and one-line explanations.
- Quick settings panel for budget, cooking time and priority.
- One personalised smart suggestion based on profile history.
- Continue button remains visible without excessive scrolling.
- Mobile layout uses a two-column intent grid and sticky Continue button.

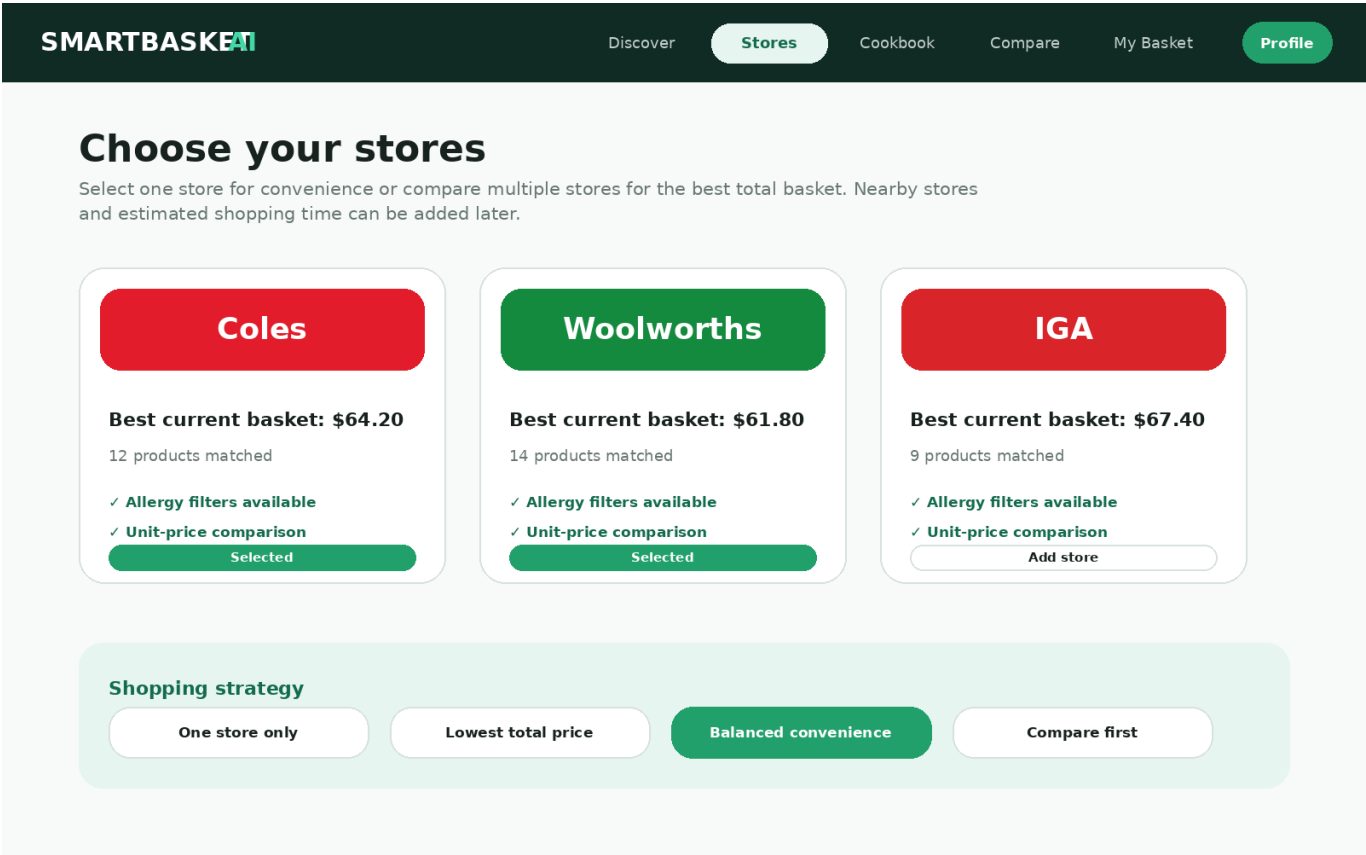


Interface example 1 - Intent-first opening experience

17.5 Store selection page specification

Store selection should feel like choosing a provider in Uber or a restaurant in a delivery app. Each card displays the store name, estimated basket price, number of matched products and available comparison features.

- Allow multi-select for Coles, Woolworths and IGA.
- Offer four strategies: one store only, lowest price, balanced convenience, or compare first.
- Show a clear Selected state and do not rely only on colour.
- Future enhancement: distance, delivery fee, stock availability and estimated shopping time.
- When one store lacks a suitable product, clearly show the best substitute from another selected store.

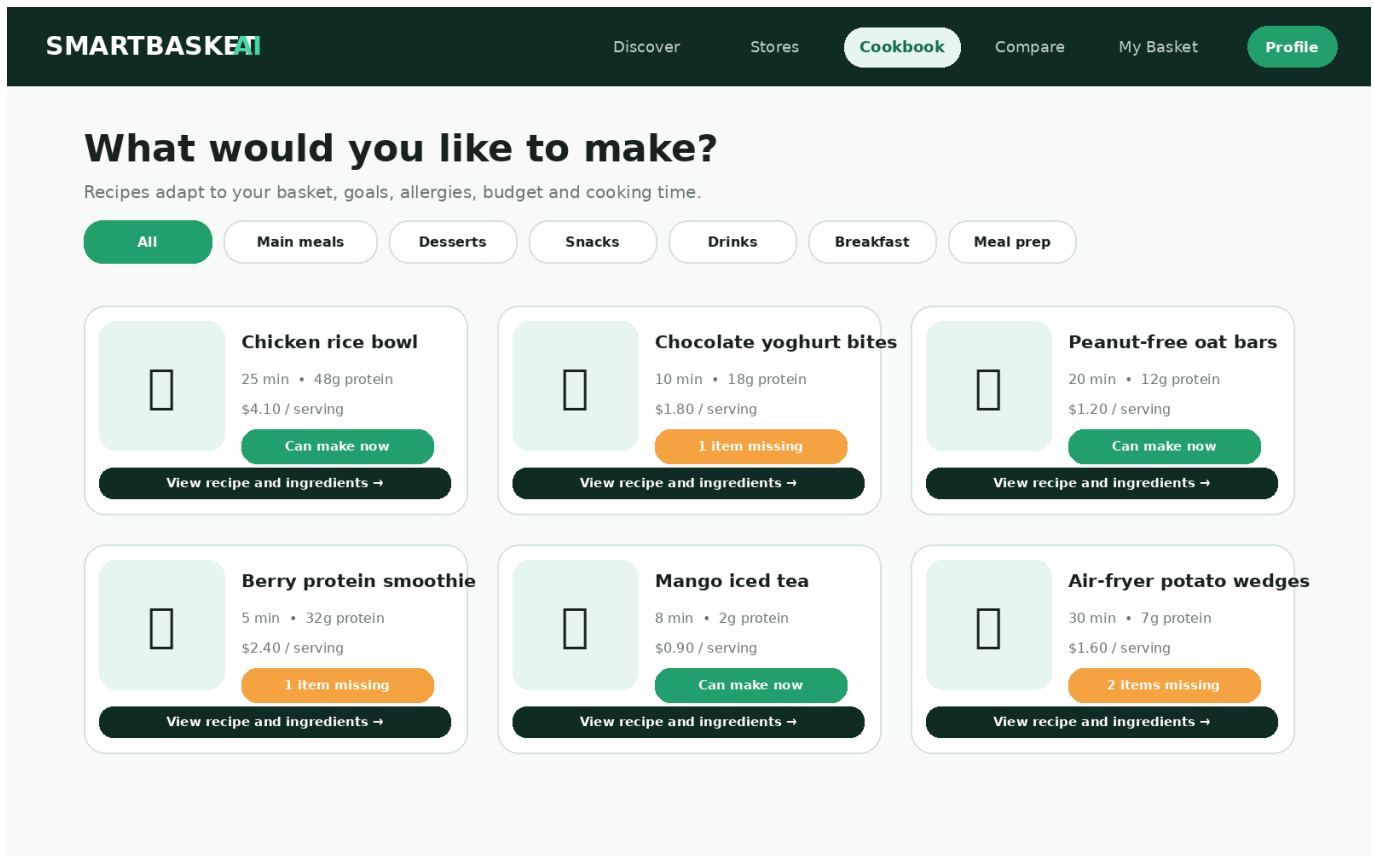


Interface example 2 - Store selection with basket-level comparison

17.6 Expanded cookbook system

The cookbook should be a central part of the product rather than a secondary recipe list. It must support complete meals, desserts, snacks and drinks, with filters that reflect the user's purpose.

Category	Examples	Useful filters
Main meals	Breakfast, lunch, dinner, meal prep	High protein, low cost, under 30 minutes
Desserts	Baked, chilled, frozen, no-bake	High-protein, lower sugar, allergy-safe
Snacks	Portable, study snacks, gym snacks	Cheap, quick, long shelf life
Drinks	Smoothies, shakes, mocktails, iced/hot drinks	Protein, hydration, caffeine-free
Use-soon	Recipes using expiring pantry items	Food-waste reduction
Can make now	Recipes requiring no new purchases	Fastest and lowest cost

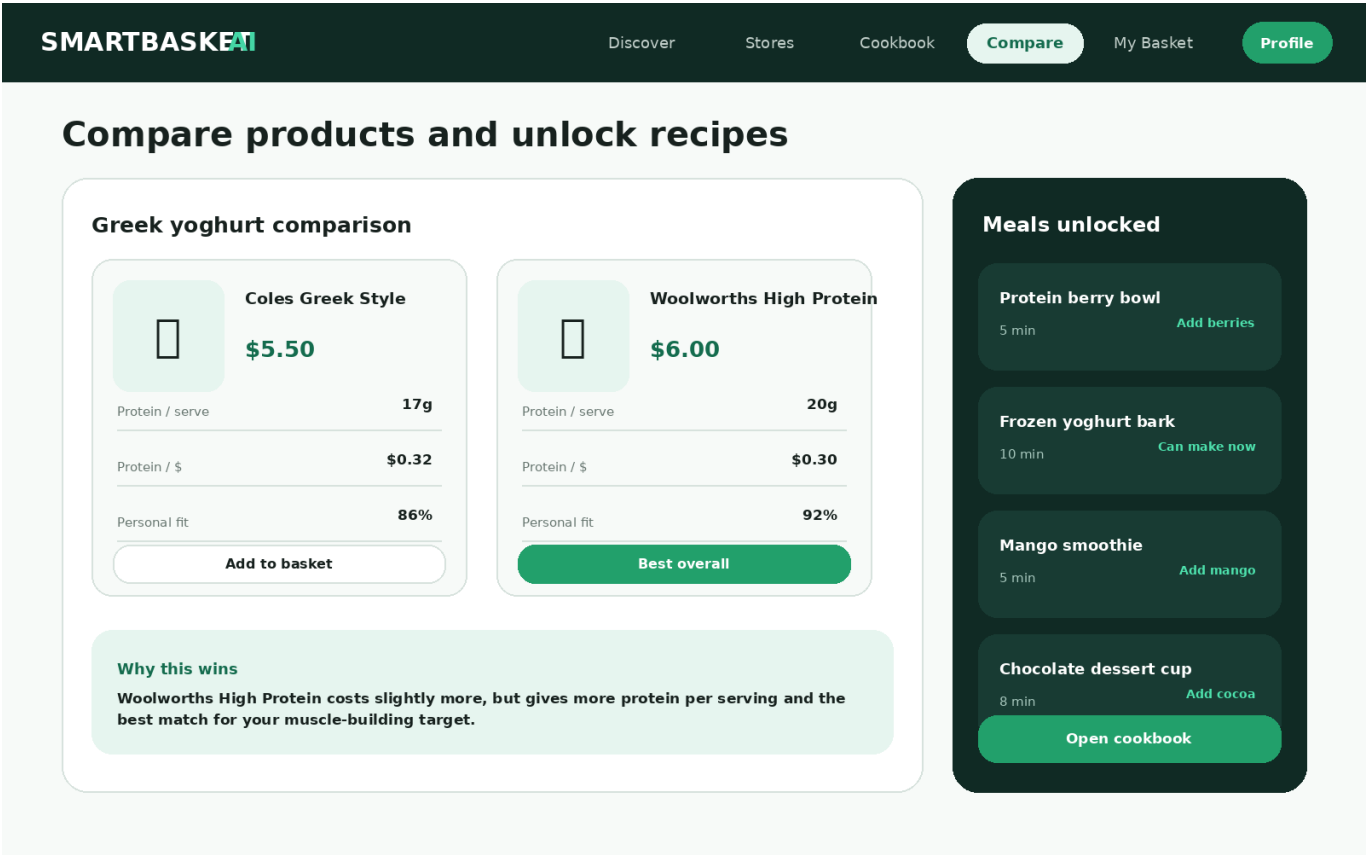


Interface example 3 - Cookbook covering meals, desserts, snacks and drinks

17.7 Product comparison and recipe side panel

The best comparison experience combines product data with an immediate explanation of what the product enables. Instead of showing a detached table, the page should show product cards on the left and newly unlocked recipes on the right.

- Highlight a winner for price, protein-per-dollar, lower sugar or sodium, and personal fit.
- Explain trade-offs in one short paragraph generated only from calculated values.
- Display recipes unlocked by the selected product.
- Allow Add to Basket directly from the comparison screen.
- Show allergy conflicts as a hard warning above the action button.
- On mobile, place the meal suggestions below the comparison cards.



Interface example 4 - Product comparison connected to recipe suggestions

17.8 Detailed screen hierarchy

Screen	Essential content	Primary action
Discover	Intent cards, quick constraints, personalised suggestion	Continue
Stores	Store cards, strategy selector, estimated basket summary	Compare selected stores
Product feed	Search, filters, product cards, recommendation reason	Add / Compare
Product detail	Nutrition, ingredients, allergens, value metrics, recipe uses	Add to basket
Compare	Side-by-side facts, winners, AI explanation, recipe unlocks	Choose product
Cookbook	Category tabs, recipe cards, missing ingredients	View recipe
Recipe detail	Ingredients, nutrition, time, steps, equipment, substitutions	Add missing items
Basket	Items, store subtotals, budget, nutrient coverage, insights	Optimise
Optimiser	Suggested swaps, savings and nutrition differences	Accept changes
Checkout summary	Final shopping list grouped by	Save / export

	store, recipe plan	
--	--------------------	--

17.9 Visual design system

- Primary colour: deep grocery green for trust and health.
- Accent colour: mint for personalised recommendations and positive states.
- Savings colour: amber/orange, reserved for budget opportunities.
- Warning colour: red only for allergens, unsafe products or serious errors.
- Use white product cards on a light neutral background.
- Use 20-30 px rounded corners and subtle borders rather than heavy shadows.
- Desktop maximum content width: approximately 1440 px.
- Use a 12-column grid on desktop, 8 columns on tablet and 4 columns on mobile.
- Minimum touch target: 44 x 44 px.
- Do not use nutrition scores without also displaying a plain-English reason.

17.10 New technical requirements caused by the expanded interface

Requirement	Implementation
Intent model	Store the selected journey type: weekly, meal, dessert, snack, drink or pantry-first.
Recipe type	Add recipe_type and occasion fields to recipes.
Store strategy	Store one_store, lowest_price, balanced or compare_first.
Recipe matching	Calculate available, missing and optional ingredients separately.
Category filters	Support cuisine, recipe type, cooking method, time, cost and nutrient goal.
Recommendation events	Record which suggestion was shown, accepted or rejected.
Mobile state	Persist current intent, selected stores and basket between pages.

17.11 Database additions

```

alter table recipes
  add column recipe_type text check (
    recipe_type in ('main', 'dessert', 'snack', 'drink', 'breakfast', 'meal_prep')
  ),
  add column cuisine text,
  add column occasion text[],
  add column equipment text[],
  add column is_no_cook boolean default false;

alter table profiles
  add column default_intent text,
  add column shopping_strategy text default 'balanced';

create table saved_journeys (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users(id) on delete cascade,
  intent text not null,
  selected_store_ids uuid[],
  constraints jsonb not null,
  created_at timestamptz default now()
);

```


17.12 Updated MVP priority

- Must: intent-first opening page.
- Must: large store-selection cards for Coles, Woolworths and IGA.
- Must: cookbook categories for mains, desserts, snacks and drinks.
- Must: product comparison connected to meal suggestions.
- Must: live basket cost, nutrition and allergy warnings.
- Should: personalised smart suggestion on the opening page.
- Should: pantry-first and use-soon modes.
- Could: nearby-store distance, delivery fees and live availability.
- Do not prioritise: payment checkout or complete retailer inventories.

17.13 Revised Claude instruction

Update the SmartBasket implementation to use an intent-first experience.

Required journey:

1. Opening page asks what the user wants: weekly groceries, main meal, dessert, snack, drink or use existing ingredients.
2. Show quick constraints for budget, cooking time and priority.
3. Show store cards for Coles, Woolworths and IGA with multi-select.
4. Let the user select one-store, cheapest, balanced or compare-first mode.
5. Cookbook must include main meals, desserts, snacks, drinks, breakfast and meal-prep categories.
6. Product comparison must display recipes unlocked by each product.
7. Use the four interface examples in the attached blueprint as the visual direction, but create reusable responsive React components.
8. Keep calculations deterministic and AI explanations grounded.
9. Implement desktop and mobile layouts.
10. Preserve all existing profile, allergy, nutrition, budget, cart, optimisation and Supabase requirements from the blueprint.

18. Solo Builder Operating Plan - Claude as the Coding Partner

This project is planned for one person. The document therefore acts as the product manager, designer and technical specification, while Claude Code or another coding agent performs most implementation work. The solo builder should avoid manually creating every component at once. Work in small, testable phases and require Claude to run, verify and repair the website before moving forward.

18.1 The solo-builder principle

- You remain the product owner: decide the scope, priorities, appearance and demo story.
- Claude is the implementation partner: generate code, create files, run commands, fix errors and document changes.
- The PDF is the source of truth. New ideas should first be added to the blueprint, then implemented.
- Never ask Claude to build the entire platform in one prompt. Use phase prompts with acceptance tests.
- Keep a working production deployment after every major phase so one broken feature cannot destroy the whole demo.

18.2 Accounts and software needed before coding

Item	Why it is needed	Recommended setup
Claude Code / Claude	Generate and edit the codebase	Use Claude Code in the project folder where possible; attach this PDF for context
GitHub	Stores the source code and version history	Create a private repository called smartbasket-ai
Vercel	Runs the public website	Connect the GitHub repository; use the free .vercel.app URL first
Supabase	Database and account login	Create one project in the Sydney or nearest practical region if available
Google Cloud Console	Google sign-in credentials	Only needed when enabling real Google OAuth
AI API account	Powers explanations or assistant features	Optional for the first interface build; add after deterministic features work
Domain registrar	Purchases the final custom address	Optional for the hackathon; buy only after confirming the name
Local tools	Run and test the website	Node.js LTS, Git, VS Code or terminal, and Chrome

18.3 Exact working method with Claude

38. Create an empty folder and GitHub repository.
39. Place this PDF in the project folder under /docs so Claude can continuously reference it.
40. Ask Claude to create PLAN.md summarising the blueprint, MVP scope and build phases.
41. Ask Claude to initialise the Next.js application and make the first commit.
42. Complete one phase at a time. Require Claude to run npm run build and report all changed files.
43. Open the local website and visually test it after each phase.

44. Push each stable phase to GitHub. Vercel will create a preview or production deployment.
45. When an error occurs, give Claude the exact terminal error, browser console error and screenshot.
46. Do not add optional features until the complete core demo works.
47. Before final submission, freeze features and spend the remaining time on testing, visual polish and the pitch.

18.4 Prompt sequence for a solo builder

Prompt 1 - Repository and planning

```
Read /docs/SmartBasket_AI_Technical_Product_Blueprint_v3.pdf.
Create PLAN.md containing:
- the final MVP
- pages and reusable components
- data models
- database tables
- build phases
- acceptance tests for each phase
Do not write application code yet. Identify any contradictions or excessive scope.
```

Prompt 2 - Create the running foundation

```
Implement Phase 1 only. Initialise Next.js App Router with TypeScript and Tailwind.
Create the global layout, navigation, landing page, intent-first opening page,
store-selection page and demo profile. Use local data only.
Run npm install, npm run lint and npm run build. Fix every error.
Then list the files created and give me the exact command to run the site.
```

Prompt 3 - Core shopping workflow

```
Implement Phase 2 only: product types, 40 realistic seed products, search, filters,
product cards, product detail, compare selection and persistent basket state.
All buttons must work. Do not add Supabase or AI yet. Run the full build and test
the main path from Discover to Basket.
```

Prompt 4 - Deterministic intelligence

```
Implement Phase 3 only: allergy checks, unit price, cost per serving, protein per
dollar, nutrition totals, personalised scores, comparison winners and basket insights.
Write unit tests for calculations. No LLM may calculate factual values.
```

Prompt 5 - Cookbook and optimisation

```
Implement Phase 4 only: recipes for mains, desserts, snacks and drinks; recipe
matching; missing ingredients; add-all-missing-items; basket substitutions; money
saved; and time-saved methodology. Preserve all existing features and run regression tests.
```

Prompt 6 - Supabase and deployment

```
Implement Phase 5 only: Supabase project integration, SQL migration files, user
profiles, saved baskets, pantry and authentication. Keep the Demo Profile working even
when authentication is unavailable. Add .env.example and update README deployment steps.
```

Prompt 7 - Grounded AI

```
Implement Phase 6 only: a server-side AI explanation endpoint. The AI receives only
structured calculated facts and cannot calculate or invent prices, nutrition, ingredients
or allergen status. Add schema validation, timeout handling, a deterministic fallback and
rate limiting.
```

19. From Blueprint to a Live Website - Exact Deployment Guide

19.1 What a domain, hosting and deployment mean

Term	Plain-English meaning	SmartBasket example
Code repository	The saved project files and history	GitHub repository smartbasket-ai
Hosting	The computers that run and serve the website	Vercel
Deployment	Publishing a version of the code online	Every push to main creates a production deployment
Domain	The human-readable website address	smartbasket.app
DNS	Records that point the domain to the host	Registrar or Cloudflare records pointing to Vercel
SSL / HTTPS	Encryption shown by the lock icon	Automatically configured after the domain is connected
Database	Persistent application data	Supabase Postgres
Environment variable	A secret or configuration stored outside code	SUPABASE_URL or AI_API_KEY

19.2 Best recommendation for the hackathon

Use the free Vercel project address first, such as smartbasket-ai.vercel.app. Vercel automatically assigns a .vercel.app domain after deployment, so a purchased domain is not required to run or demonstrate the website. Add a custom domain only after the application is stable and the product name is final.

19.3 Step-by-step: make the website run locally

48. Install Node.js LTS and Git.
49. Open Terminal and enter the project folder.
50. Run npm install to download the project dependencies.
51. Copy .env.example to .env.local and add the required development values.
52. Run npm run dev.
53. Open http://localhost:3000 in Chrome.
54. Test Discover, Stores, Cookbook, Compare and Basket.
55. Run npm run build before publishing. A local development page working is not enough; the production build must pass.

```
npm install
cp .env.example .env.local
npm run dev
# open http://localhost:3000

# before deployment
npm run lint
npm run test
npm run build
```

19.4 Step-by-step: deploy with GitHub and Vercel

56. Create a GitHub repository and push the project to the main branch.

57. Sign into Vercel using GitHub.
58. Choose Add New Project and import smartbasket-ai.
59. Confirm that Vercel detects Next.js.
60. Add environment variables in Vercel Project Settings. Never paste secret keys into source files.
61. Select Deploy.
62. Open the generated .vercel.app address and test the production site.
63. Every later push to main will trigger a new production deployment. Branches or pull requests can create preview deployments.
64. Keep the latest known-good Vercel deployment available in case a new change breaks the project.

19.5 Step-by-step: choose and connect a custom domain

65. Choose three to five possible names. Check spelling, pronunciation, social handles and possible trademark conflicts.
66. Prefer a short .com, .app, .ai or Australian-relevant option. A domain is branding, not a technical requirement.
67. Search availability at a registrar such as Cloudflare Registrar or through another reputable registrar.
68. Register the domain using accurate contact information and enable account two-factor authentication.
69. In Vercel, open the project, then Settings > Domains > Add Domain.
70. Enter the domain and follow the DNS instructions shown by Vercel.
71. If the registrar manages DNS, add the exact A, CNAME or nameserver records that Vercel requests.
72. Wait for verification and HTTPS activation.
73. Choose whether www redirects to the root domain or the root redirects to www.
74. Add the final production domain to Supabase authentication redirect URLs and Google OAuth authorised redirect URLs.
75. Test login, password reset, API requests, images and mobile behaviour on the custom domain.

19.6 Domain-name checklist

- Easy to spell after hearing it once.
- No confusing hyphens or unnecessary numbers.
- Broad enough to include groceries, recipes, snacks and drinks.
- Not dependent on one supermarket brand.
- Available as a domain and preferably as a social handle.
- Does not closely imitate an existing food, health or delivery brand.
- Still makes sense if the application expands beyond the hackathon.

19.7 Suggested naming direction

Direction	Examples to investigate	Positioning
Smart basket	SmartBasket, BasketWise, BasketPilot	Grocery decision assistant
Budget and nutrition	NutriCart, MacroBasket, ValueFuel	Fitness and value
Cooking outcome	CartToCook, BasketToBite, MealUnlock	Turns shopping into meals
Personal assistant	PantryPilot, FoodFlow, ShopChef	End-to-end planning

These are naming directions, not confirmed available domains. Check domain and trademark availability before selecting one.

20. Supabase Setup - Database, Login and Persistent Data

20.1 Create the project

76. Create a Supabase organisation and project.
77. Record the project URL and publishable/anon key.
78. Use the SQL Editor or migration files to create the tables from this blueprint.
79. Enable Row Level Security for every table containing user-owned data.
80. Insert seed stores, products, nutrition and recipes.
81. Create a test user and verify that one user cannot access another user's cart or pantry.
82. Add the Supabase values to .env.local and Vercel environment variables.

20.2 Recommended environment variables

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
SUPABASE_SERVICE_ROLE_KEY=          # server only
AI_API_KEY=                          # server only
NEXT_PUBLIC_APP_URL=http://localhost:3000
DATA_MODE=seed                      # seed | external | hybrid
```

20.3 Google sign-in setup

83. Create OAuth credentials in Google Cloud Console.
84. Configure the consent screen and authorised domains.
85. Add the callback URL shown in Supabase Auth provider settings.
86. Enable Google in Supabase Auth providers and add the client ID and secret.
87. Add both localhost and production redirect URLs in Supabase.
88. Test new sign-up, returning sign-in, logout and expired sessions.

20.4 Mandatory fallback for the hackathon

Always keep a “Use Demo Profile” button that loads a complete local or seeded experience without Google sign-in. Authentication providers or network services can fail during judging. The core demonstration must remain usable.

21. Data Acquisition and Reliability Plan

21.1 Recommended data mode for one person

Use hybrid data. The interface and demo should rely on a validated curated seed dataset, while an optional import script can enrich nutrition or barcode information from public sources. Do not make the entire live demo dependent on retailer scraping or an external API.

Mode	Benefits	Risk	Recommendation
Seed only	Reliable, fast and controllable	Limited catalogue and prices	Best for first working version

External API only	Broader data	Missing fields, rate limits and outages	Do not use as the only source
Hybrid	Reliable demo plus optional enrichment	More implementation work	Best final hackathon architecture

21.2 Minimum seed dataset

- 60 products: approximately 20 per store or comparable groups across stores.
- At least 8 categories: protein, dairy, grains, produce, pantry, snacks, desserts and drinks.
- At least 24 recipes: 8 mains, 5 desserts, 5 snacks and 6 drinks/breakfast options.
- At least 5 intentional product comparison pairs.
- At least 3 allergen-conflict examples.
- At least 5 optimiser substitutions with clear savings.
- All prices marked with a captured date and demo-data label when not live.

21.3 Data validation rules

- Price, package size and serving size must be positive numbers.
- Nutrition must clearly identify whether it is per serving or per 100 g.
- Unknown values remain null; never convert missing values to zero.
- Allergens use a controlled vocabulary and aliases.
- Store price records include a captured_at timestamp.
- Recipe nutrition is calculated from ingredient data where possible, not invented by the AI.
- Product images require a fallback placeholder and source permission.

22. Production Readiness, Cost Control and Failure Handling

22.1 What must work without AI

- Opening, store selection, search, filters and navigation.
- Cart calculations, comparison values and allergen warnings.
- Recipe matching from structured ingredients.
- Basket optimisation and savings calculations.
- Demo profile and seed products.
- Final shopping list grouped by store.

22.2 AI cost and reliability controls

- Call the model only when the user requests an explanation or generated variation.
- Cache repeated explanations for identical comparison inputs.
- Limit output length and provide structured context.
- Rate-limit the endpoint per user or IP.
- Add a 10-20 second timeout and deterministic fallback text.
- Track request count during the hackathon to prevent unexpected usage.

22.3 Failure matrix

Failure	What the user sees	Required fallback
---------	--------------------	-------------------

AI API unavailable	Short service notice	Template explanation from calculated facts
Supabase unavailable	Demo mode message	Local seed data and in-memory basket
External product API fails	Last-updated notice	Curated local products
Product image fails	Neutral food placeholder	Never break the card layout
Google login fails	Try again and demo option	Use Demo Profile
Optimiser finds no safe swap	No safe alternative found	Keep current item and explain why

23. Complete Acceptance Checklist for Claude

Area	Definition of done
Project	npm install and npm run build complete successfully
Navigation	Every navigation item opens a real page
Opening	The user can choose groceries, main, dessert, snack, drink or pantry-first
Stores	Coles, Woolworths and IGA cards support selection
Products	Search, filters, details, compare and cart work
Safety	Allergen conflicts are visible and excluded from automatic recommendations
Cookbook	Mains, desserts, snacks and drinks are represented
Recipes	Available and missing ingredients update from the basket
Optimiser	At least one swap changes cost and explains the trade-off
Savings	Money and time methodology are visible
Auth	Demo mode always works; real auth works when configured
Database	User-owned records use Row Level Security
AI	No factual calculations are delegated to the LLM
Mobile	Core journey works at phone width
Deployment	Production URL works in an incognito browser
Recovery	A backup screen recording and known-good deployment exist

24. Updated Final Recommendation

For one person using Claude, the smartest path is not to buy a domain or integrate every service first. Build a complete local experience using seed data, deploy it to the free Vercel address, and only then add Supabase, Hackathon implementation guide • 32

Google sign-in, grounded AI and a custom domain. The strongest demonstration is a reliable full journey, not the largest list of unfinished features.

24.1 Recommended order of priority

89. Polished interface and complete navigation.
90. Reliable seeded product and recipe data.
91. Working comparison, cart and cookbook.
92. Allergy, budget and nutrition calculations.
93. Basket optimiser and measurable savings.
94. Public Vercel deployment.
95. Supabase persistence and optional Google authentication.
96. Grounded AI explanations.
97. Custom domain and final branding.

24.2 Current official technical references

Reference	What it confirms
Vercel - Working with domains: https://vercel.com/docs/domains/working-with-domains	A deployment receives a .vercel.app address; custom domains can be added later.
Vercel - Add and configure a domain: https://vercel.com/docs/domains/working-with-domains/add-a-domain	Custom domain setup is managed from Project Settings > Domains and requires DNS verification.
Vercel - Next.js: https://vercel.com/docs/frameworks/full-stack/nextjs	Next.js projects can be imported or deployed with Vercel tooling.
Supabase - Next.js quickstart: https://supabase.com/docs/guides/getting-started/quickstarts/nextjs	A Supabase project can provide Postgres data access for a Next.js application.
Supabase - Auth with Next.js: https://supabase.com/docs/guides/auth/quickstarts/nextjs	Supabase documents authentication setup for the Next.js App Router.
Cloudflare Registrar: https://developers.cloudflare.com/registrar/	Cloudflare supports domain registration and DNS management; registrar restrictions should be reviewed.
Claude Code overview: https://docs.anthropic.com/en/docs/claude-code/overview	Claude Code can understand a codebase, build features, fix bugs and automate development workflows.

Version 3 - expanded for a solo builder, live deployment, domains, Supabase setup, Claude workflow and production reliability.